

AI and Robotics

Lesson 1b: Approaches to AI Decision-Making

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Table of contents

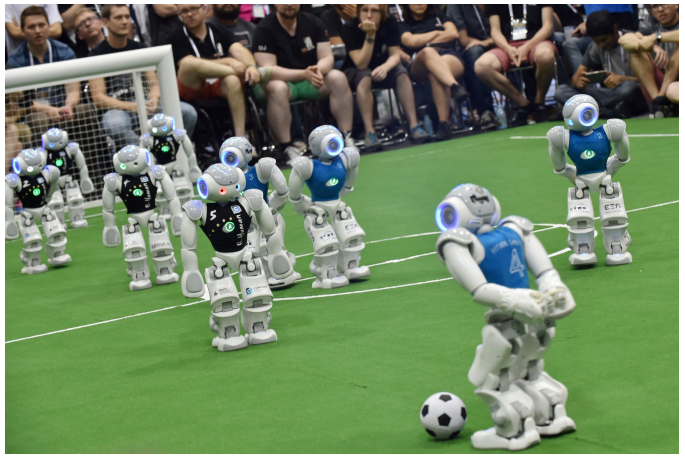
1. Autonomy

2. Approaches to AI Decision Making

3. Formal Models

Autonomy





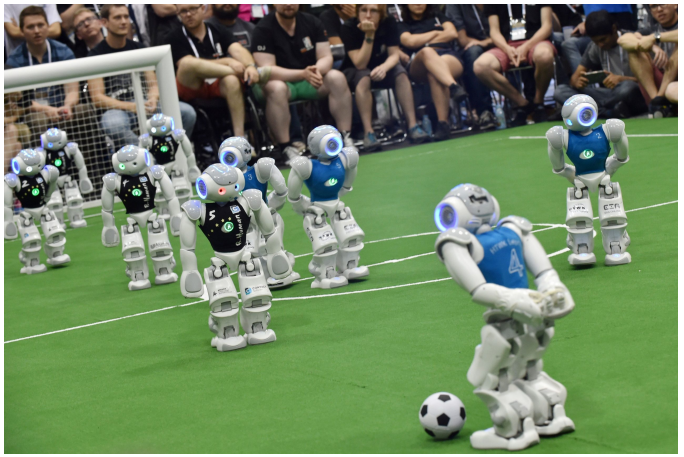


Reactive Agents

Fixed policy based on sensor input

No internal model of the world

Fast but limited adaptability



Rational Agents

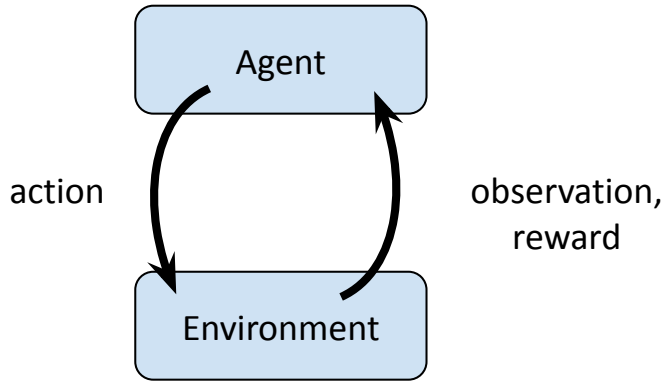
Use planning and reasoning

Represent world knowledge

Flexible but computationally costly

Autonomy



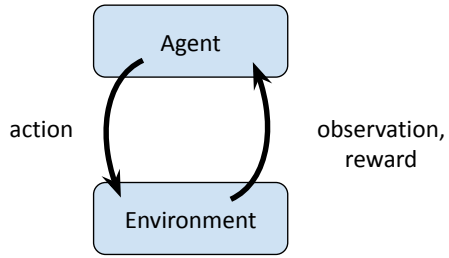


Autonomy
Approaches to AI Decision Making
Formal Models

Autonomy

- Action (Actuation)
  - Locomotion
  - Manipulation
- Perception (Sensors)
  - Internal
  - External
- Cognition (Control)
  - From reactive to proactive
  - From finite state machines to cognitive robotics





Autonomy
Approaches to AI Decision Making
Formal Models

Bluey and Bingo



https://youtu.be/n4rh2jD8OkY

Autonomy
Approaches to AI Decision Making
Formal Models

Approaches to AI Decision Making

Approaches to AI Decision Making

We are seeking a **policy**, a (potentially probabilistic) mapping from the state space to actions.

Basic approaches:

- **Programming/reactive** - fixed decision making
- **Planning** - using a domain-independent descriptive language
- **Machine Learning** - extracting a policy from data and experience
- **Reinforcement Learning** - learning by interacting with the environment

Example

You are in Kibutz Ein Dor and you want to get to Austin Texas.





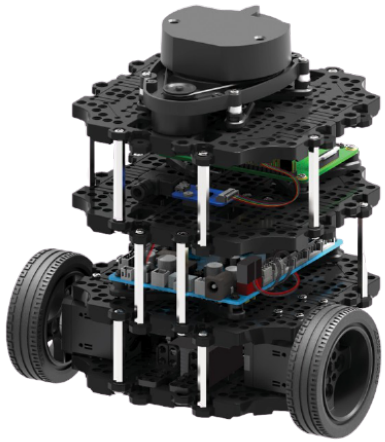


How do you decide how to get to Austin?

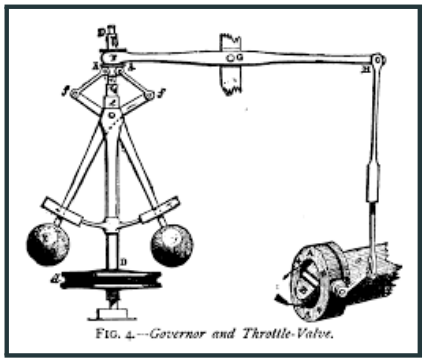
Other examples



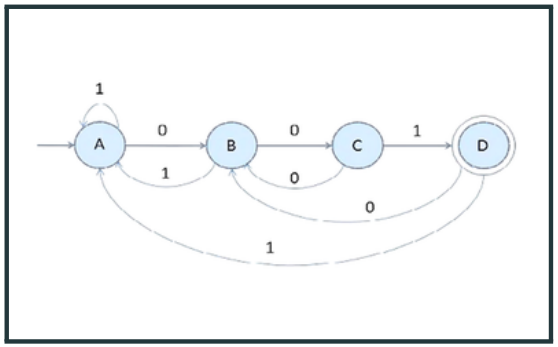


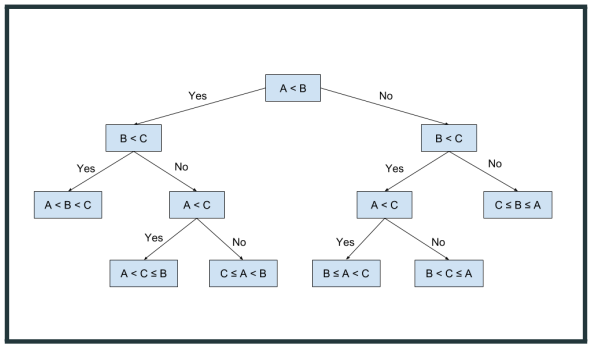


Programming/reactive









Very practical in many applications, but lacks flexibility.

Is this approach relevant to robotics ?

Planning

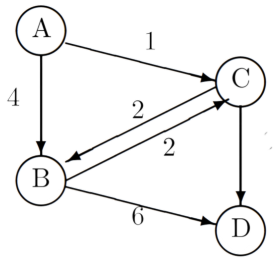


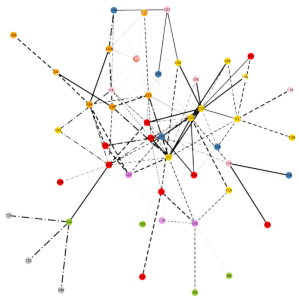
Planning is the **model-based** approach to autonomous behavior.

The same tools work for many problems: block stacking, cooking, logistics, Mars rovers, cyber security, text generation, robotics, ...

Planning as State Space Search

- A system can be in one of many **states**
- States assign values to a set of **variable** (features).
- **Actions** change the values of certain variables.
- **Reward** is a numeric signal passed from the environment to the agent - used to signal the objective.
- **Goal condition** a condition over the state space that specifies the goal states.





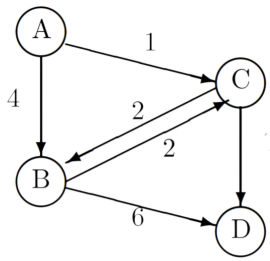
Planning as State Space Search

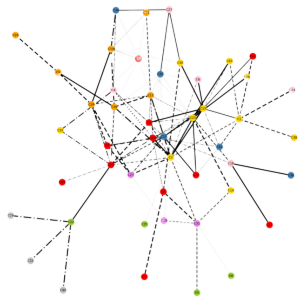
Basic task: find a **policy** to

- drive the initial state into a goal state
- maximize the expected accumulated reward

Language is **generic** and not domain specific

Complexity: classical planning is PSPACE-complete - may require exponential time but polynomial space (BUT space is exponential in the number of state variables).





Planning as State Space Search

- **Classical planning** - fully observable deterministic
- **Probabilistic planning** - fully observable stochastic action outcomes
- **Planning with partial information** - partially observable stochastic action outcomes

Which planning algorithms do you know ?

Planning as State Space Search

- **Classical planning** - fully observable deterministic
- **Probabilistic planning** - fully observable stochastic action outcomes
- **Planning with partial information** - partially observable stochastic action outcomes

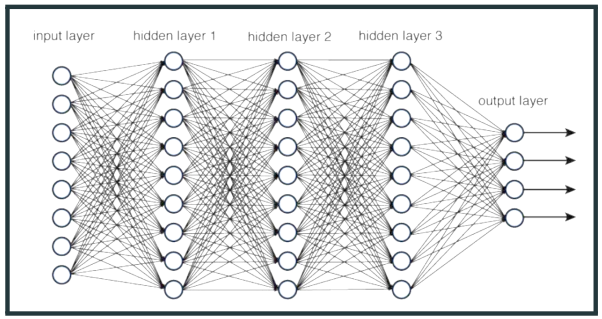
Which planning algorithms do you know ?

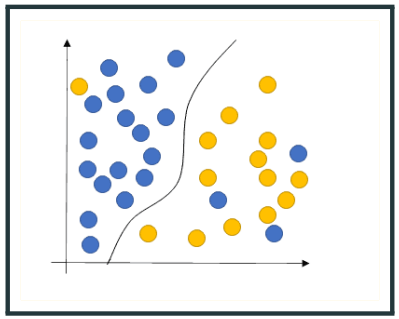
We will re-visit some planning algorithms you already know (A*, policy iteration, value iteration, MCTS) and explore new approaches (LAO*, planning in belief space).

Machine Learning

Use statistics to find patterns in massive amounts of data.

The input vector x (e.g., an image) is mapped to an output $f(x)$, which represents a classification label, a probability distribution over the possible labels, or object labels with suitable bounding boxes.





Machine Learning

Template algorithm (for supervised learning)*:

- **Input:**
  - Sample set: $S = \{(x_i, y_i)_{i=1}^m\}$
  - Model calls H (candidate classifiers)
- **Output:**
  - Classifier with lowest **expected error**
  - For example, error of model $h \in H$ measured via **loss** measured as Mean Squared Error (for regression):

$$L(h(x), y) = (h(x) - y)^2$$

Various approaches and methods for learning: linear regression, decision trees, DNN etc.

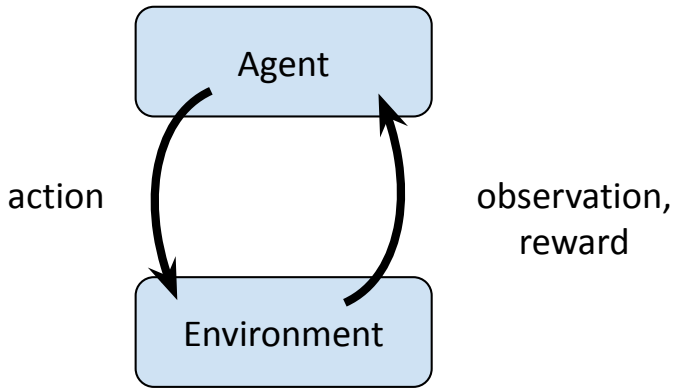
Machine Learning: Imitation Learning

- Learning from demonstrations
- Useful when an expert can demonstrate the desired behaviour (rather than to specify a reward function).
- Simplest form is behaviour cloning (BC), which focuses on learning the expert's policy using supervised learning.



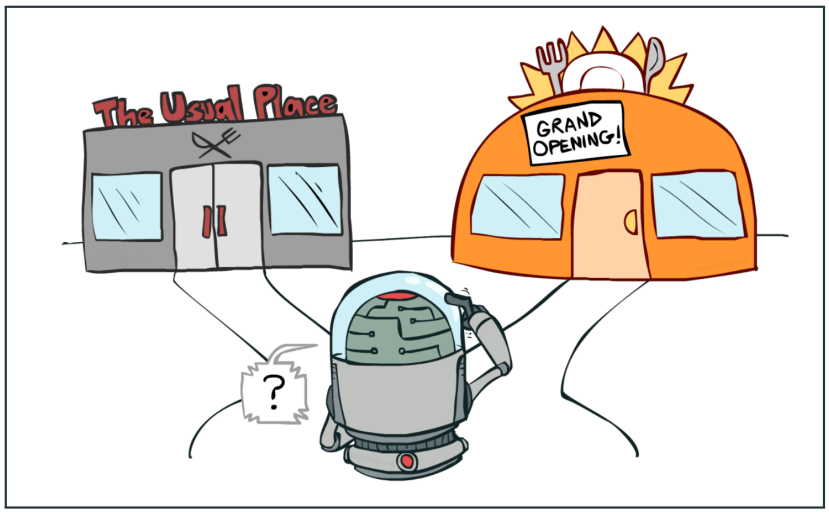
Reinforcement Learning (RL)

- Learning from interaction how to achieve a goal.
- The learner is not told which actions to take, but instead must discover which actions yield the highest total **reward** by trying them.



Reinforcement Learning (RL)

In RL there's a tradeoff between **exploration** (choosing an action for which the outcome is unknown or choosing a random action) and **exploitation** (choosing an action based on learned values).

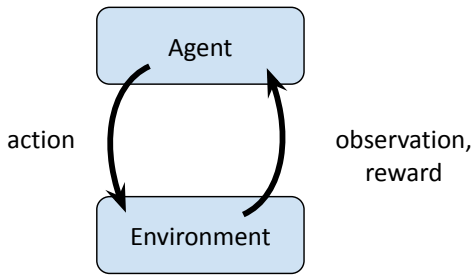


Reinforcement Learning (RL)

The process involves:

- Observing of the environment
- Deciding how to act (using some policy)
- Executing the chosen action
- Receiving a reward (or penalty) and observation
- Learning from the experiences and refining the policy

This iterative process continues until convergence or until resources are exhausted



RL vs. Planning

RL vs. Planning

Both are fundamental problems in sequential decision making

- **Planning:** A model of the environment is known. The agent performs computations with its model (without any external interaction). The agent improves its policy a.k.a. deliberation, reasoning, introspection, pondering, thought, and search.
- **Reinforcement Learning:** The environment is initially unknown. The agent improves its policy by interacting with the environment.

The distinction is very fuzzy...

RL vs. ML

What makes reinforcement learning different from other machine learning paradigms?

RL vs. ML

What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor, only a reward signal
- Feedback is delayed, not instantaneous
- Time really matters (sequential, non i.i.d data): agent's actions affect the subsequent reward it receives

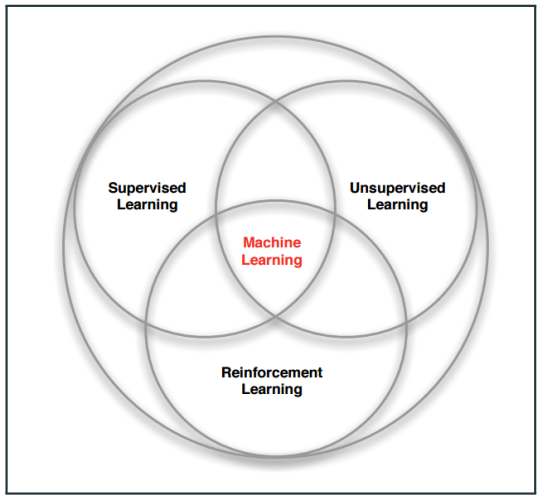
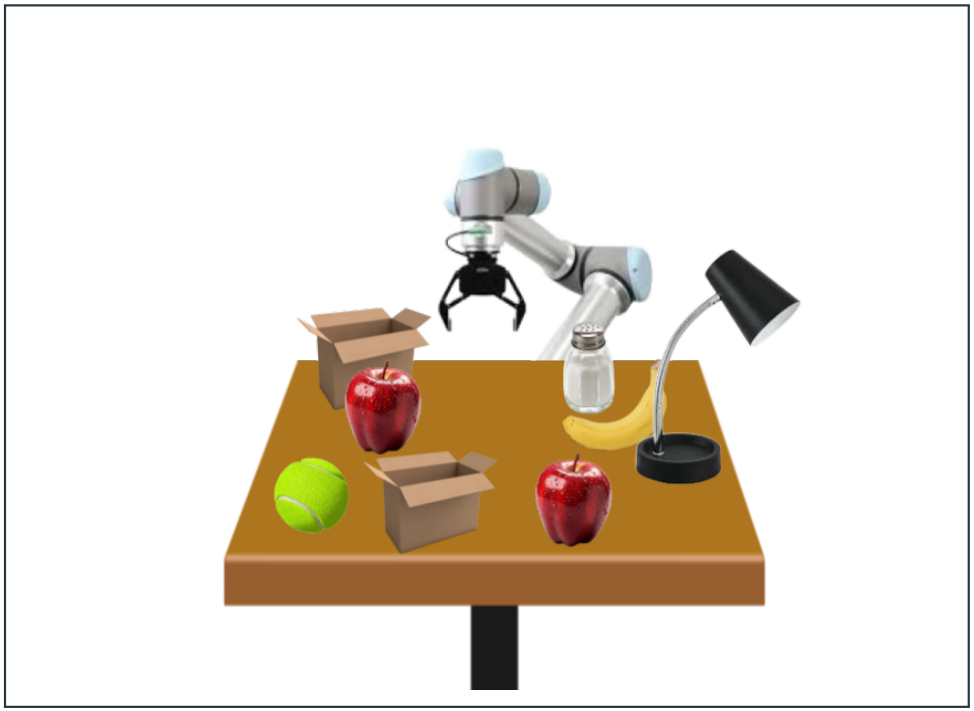


Image by David Silver.

Autonomy
Approaches to AI Decision Making
Formal Models

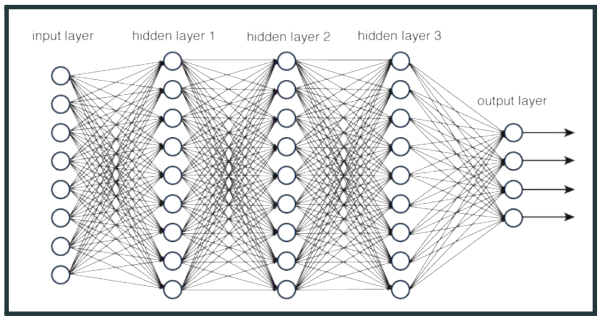
Best approach for our setting?

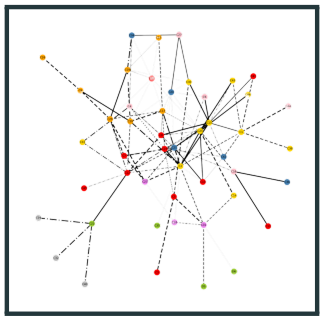


Formal Models

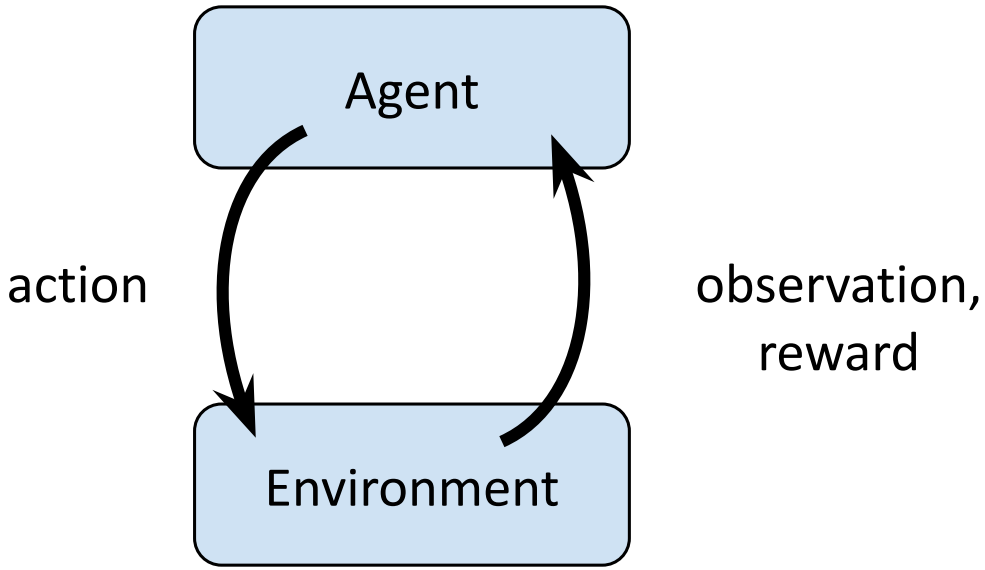
Model is an overloaded term

- In ML:
  - model is used to represent the structure of the network used for classification / regression.
  - is part of the output of the algorithm
- Here:
  - models denotes the representation / abstraction of the environment.
  - is part of the input





Modeling the Environment



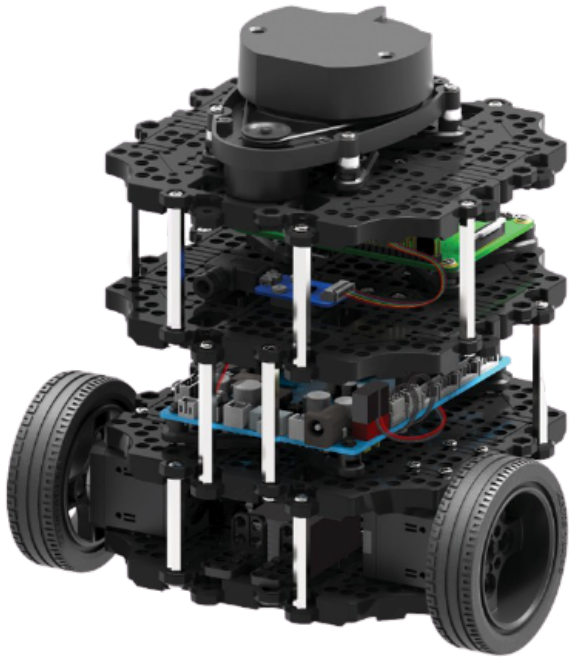
What do we need to model ?

The model needs to be compact enough to be solvable but informative enough to be useful.

Environment: State and action space

- **State Space:** $s \in \mathcal{S}$ is a representation of a world state
  - Typically, it is impossible / impractical to explicitly maintain the state space.
  - We therefore use a **factored representation** : a state is described via a set of variables $\mathcal{X} = X_1, \ldots, X_n$. A state is an assignment of a value $X_i \in Dom(X_i)$ for each variable X_i.
- **Action space:** $a \in \mathcal{A}$ is an action an agent can perform
  - can have deterministic / non-deterministic / stochastic effects
  - may be associated with preconditions and effects.

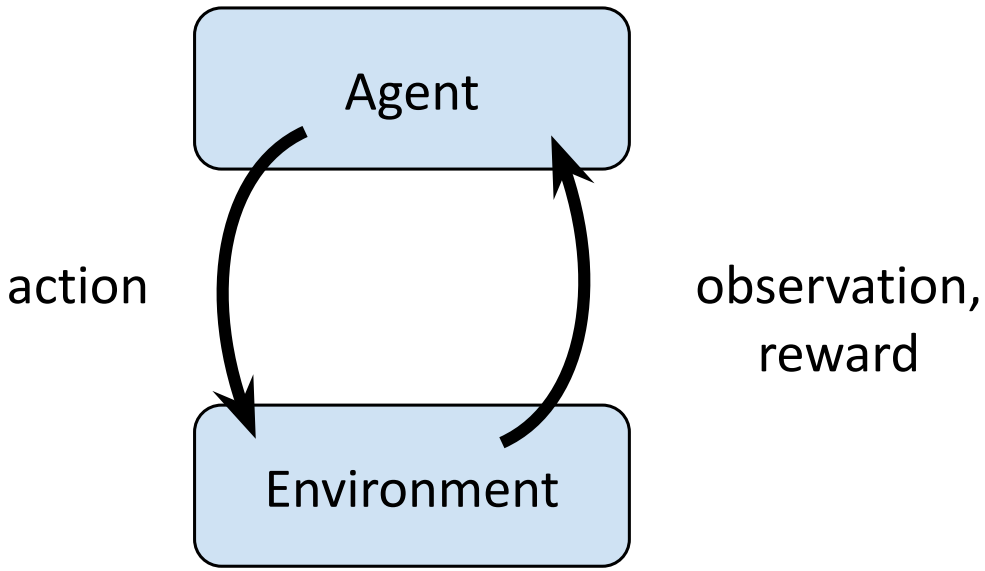
Consider a robot that is navigating in a mapped environment and needs to get from its current location A to location B.



What are the states and actions ?

The Agent-Environment Interface

- An agent operates in the environment by taking actions.
- An agent and its behavior are characterized by its:
  - **Objectives**
  - Ability to **sense** the environment and interpret the observation



The Agent-Environment Interface

At each time step $t = 1, 2, 3, ...$

- The agent receives some representation of the environment's state
  - under full observability: $s_t \in \mathcal{S}$
  - under partial observability: $o_t \in \Omega$
- On that basis selects an action $a_t \in \mathcal{A}(s)$

One time step later $t + 1$:

- The agent receives a numerical reward $r_{t+1} \in R$
- Finds itself in a new state s_{t+1}

A **trajectory**:

$$\tau = s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, ...$$

Under partial observability, the agent perceives a **history**:

$$h = o_0, a_0, r_1, o_1, a_1, r_2, o_2, a_2, r_3, ...$$

Reward

- A numeric signal passed from the environment to the agent (**cost** when negative).
- Depending on the setting, defined as $\mathcal{R}(s, a, s')$, $\mathcal{R}(s, a)$ or $\mathcal{R}(s)$
- An agent's objectives is to maximize it's **utility** $\mathcal{U}$, typically defined as the expected total reward it receives

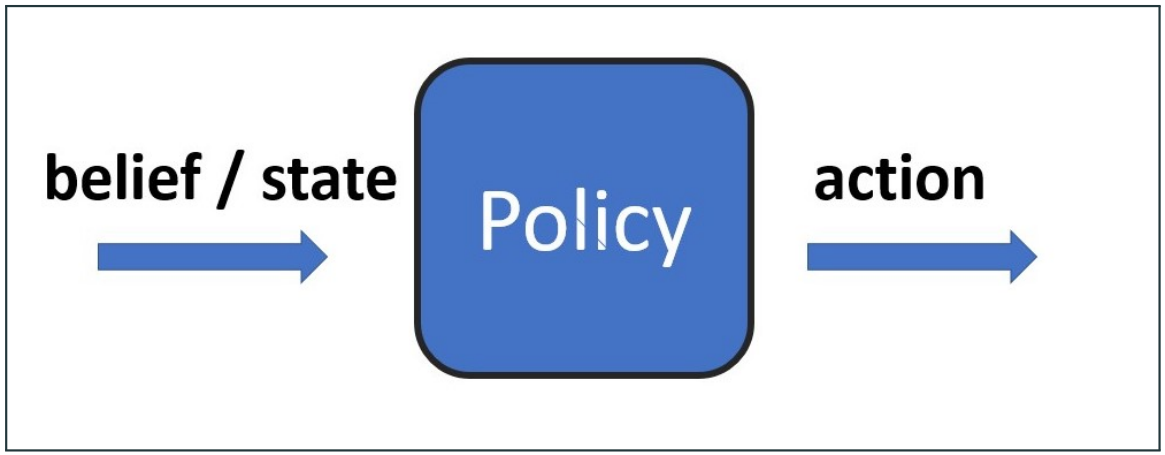
Observation and Belief




- At each time step the agent receives an observation o_t that reflects the current state of the world.
- A **belief** $\beta \in \mathcal{B}$ represents the possible world states (i.e., the states that are deemed possible by the agent).
- Induced by an **observability/ sensor function** that maps states to observations (more on this later)

Policy (for a single agent[1])

- A policy is a mapping from states to actions.
- More accurately: we seek a mapping from the agent's understanding of the state, i.e., its **belief**, to actions.
- Our objective is to find a **(sub)-optimal policy** for an agent to follow in order to achieve it's objective / maximize its **utility**.



[1] we will consider multi-agent policies later in the course

Policy

- A **deterministic** policy π is a mapping:
  - $\pi : \mathcal{S} \to \mathcal{A}$ or $\pi : \mathcal{B} \to \mathcal{A}$
  - $\pi(s)$/ $\pi(\beta)$ is the (single) action the agent will perform at state s or belief β.
- A **non-deterministic** policy is a mapping:
  - $\pi : \mathcal{S} \to \mathcal{A}^n$ or $\pi : \mathcal{B} \to \mathcal{A}^n$
  - $\pi(s)$/ $\pi(\beta)$ is the set of actions one of which the agent will perform at state s or belief β.
- A **stochastic** policy is a mapping:
  - $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ or $\pi : \mathcal{B} \times \mathcal{A} \to [0, 1]$
  - $\pi(s, a)$/ $\pi(\beta, a)$ is the probability the agent will perform action a at state s/ belief β.

Supervised Learning

In supervised learning:

- A sample set: $S = \{(x_i, y_i)_{i=1}^m\}$
- Typically used to learn for an appropriate classifier (e.g., with lowest expected error or loss) among a hypothesis class H.

Learning a policy π from data:

- A sample set: $S = \{(s_i, a_i)_{i=1}^m\}$ or $S = \{(\beta_i, a_i)_{i=1}^m\}$
- Typically, a parameterized policy over a factored state-space representation is used.
- Parameterized policy as a conditional probability $\pi_\theta(a_t|\beta_t)$ or $\pi_\theta(a_t|s_t)$



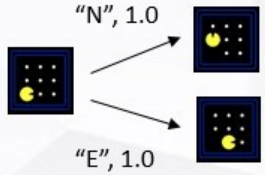
Classical Planning (from IAI)

Search Problems

- A **search problem** consists of:
  - A **state space**



  - A **successor** function (with actions, costs) or a set of **operators**



  - A **start state** and a **goal test**
- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state

Introduction to AI 236501 | TECHNION | The Henry and Marilyn Taub Faculty of Computer Science

Propositional STRIPS

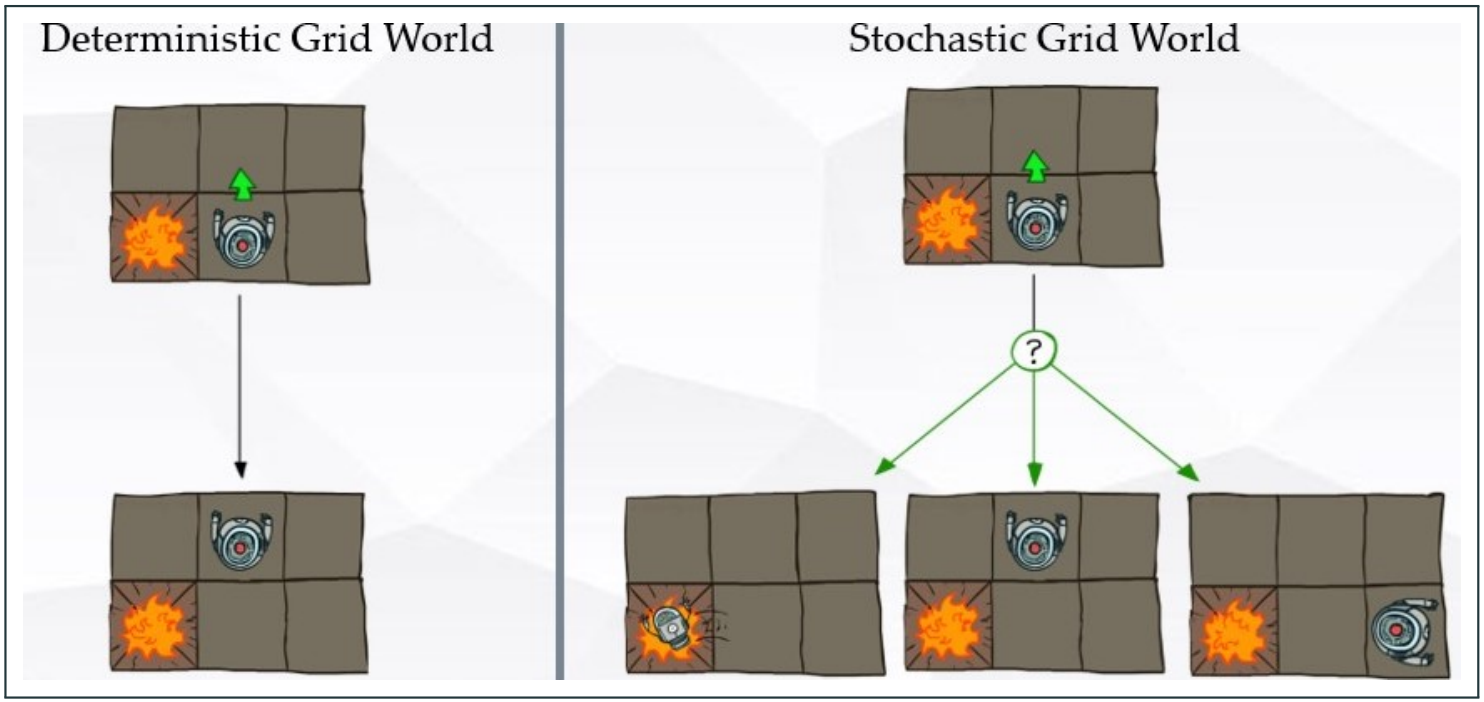
Propositional STRIPS planning task defined by a tuple $P = \langle \mathcal{F}, I, \mathcal{A}, G, \mathcal{C} \rangle$, where

- $\mathcal{F}$ are fluents, and a state s is a truth assignment over $\mathcal{F}$:
- $I \subseteq F$ is the initial state
- $G \subseteq F$ represents the set of goal states, and
- $\mathcal{A}$ is a set of actions.
  - each action is a triple $a = \langle pre(a), add(a), del(a) \rangle$, which represents the precondition, add, and delete lists respectively, all subsets of $\mathcal{F}$.
  - an action a is applicable in state s if $pre(a) \subseteq s$.
  - when a is applied in state s, it results in a new state $s' = (s \setminus del(a)) \cup add(a)$.
- $\mathcal{C} : \mathcal{A} \to \mathbb{R}_0^+$ is a cost function that assigns each action a non-negative cost.

Propositional STRIPS

- The objective is to find a **plan** $\pi = \langle a_1, \ldots, a_n \rangle$ - a sequence of actions that brings an agent from the initial state to a goal state.
- Typically, the objective is to find an optimal plan π^* that minimizes the associated cost $\mathcal{C}(\pi) = \Sigma_{i=1}^{n}(\mathcal{C}(a_i))$.
- The literature is rich with different approaches developed to solve the planning problem (Bonet and Geffner 2013)

From Deterministic to Stochastic Domains (from IAI)



Markov Decision Process

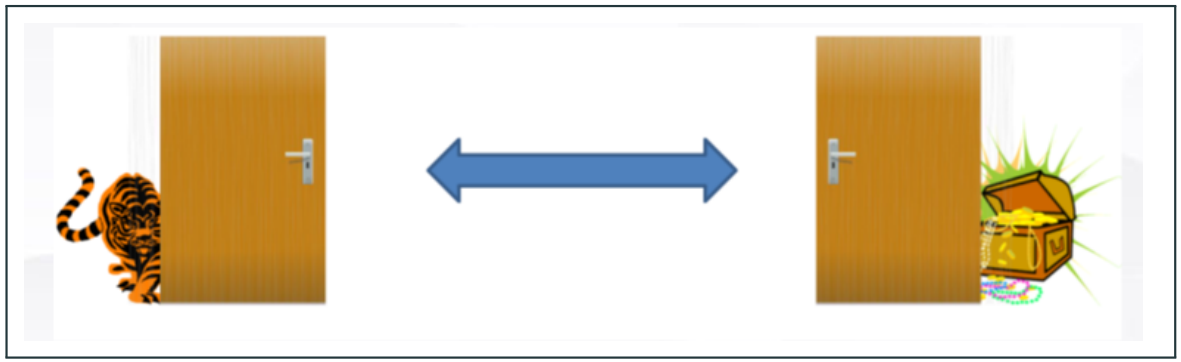
A **Markov Decision Process** (MDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ where

- $\mathcal{S}$ is a finite set of states (states are described via a set of random variables $\mathcal{X} = X_1, \ldots, X_n$).
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the **transition probability function**, where $\mathcal{P}(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$ denotes the probability of transitioning to state s' after taking action a in state s.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the **reward function**, where $\mathcal{R}(s, a)$ (or its expectation) gives the immediate reward obtained after taking action a in state s, and
- **optional:** γ is a discount factor $\gamma \in [0, 1]$ that is used to favor immediate rewards over future rewards.

The Markov property: "The future is independent of the past given the present".

Extensions: Infinite and continuous MDPs, partially observable MDPs, undiscounted, average reward MDPs. etc.

Accounting for Partial Observability (from IAI)



Belief

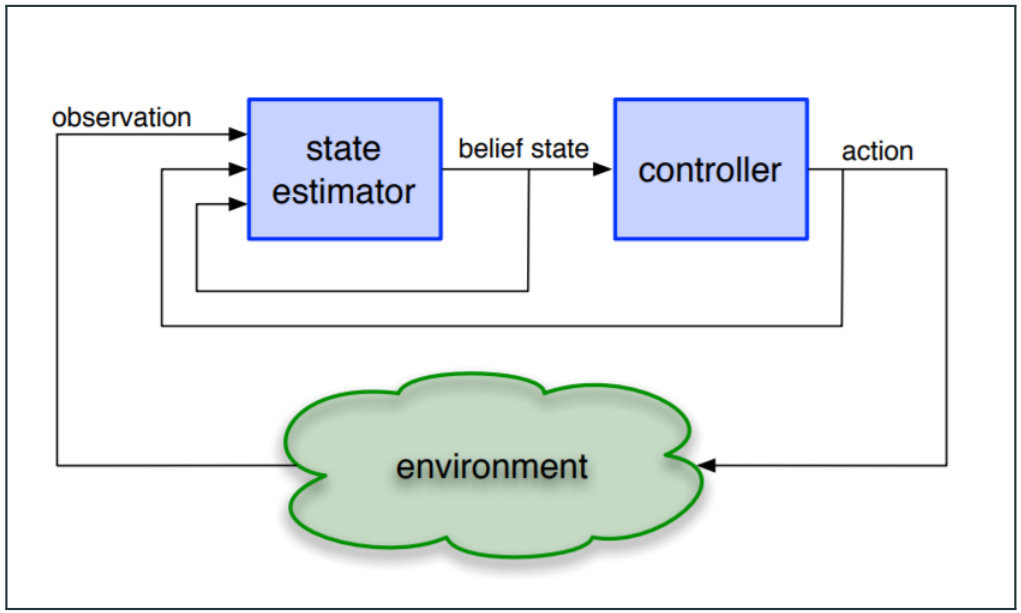
A belief $\beta \in \mathcal{B}$ can have different representations:

- **Deterministic:** a direct mapping from states to beliefs $\beta = s$ (in which case we simply refer to states)
- **Non-deterministic:** $\beta \subseteq \mathcal{S}$ represents the set of possible world states
- **Stochastic:** the belief is a probability distribution over possible underlying world states. $\beta : \mathcal{S} \to \mathcal{P}[\mathcal{S}]$ such that $\beta(s)$ represents the probability that s is the actual world state.

A partially informed agent receives **history**:
$h = o_0, a_0, r_1, o_1, a_1, r_2, o_2, a_2, r_3, ...$ and needs to maintain its belief.

Belief Tracking

- The process by which an agent maintains its belief β
- Typically relies on a **sensor function** that maps world states to observations



From Kaelbling, L. P., and T. Lozano-Perez. "Integrated Task and Motion Planning in Belief Space" 2013

Partially Observable Markov Decision Process (POMDP)

A **Partially Observable Markov Decision Process**(POMDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \Omega, \mathcal{O}, \beta_0 \rangle$ where

- $\mathcal{S}$, $\mathcal{A}$, $\mathcal{P}$, $\mathcal{R}$ and γ are as for an MDP.
- Ω is a set of observations (observation tokens),
- $\mathcal{O} : \Omega \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is a sensor function specifying the conditional probability $\mathcal{O}(o|s, a) = p(O_{t+1} = o|S_t = s, A_t = a)$ of receiving observation $o \in \Omega$ in state s after applying a [2].
- β_0 the initial belief: a probability distribution over the states.

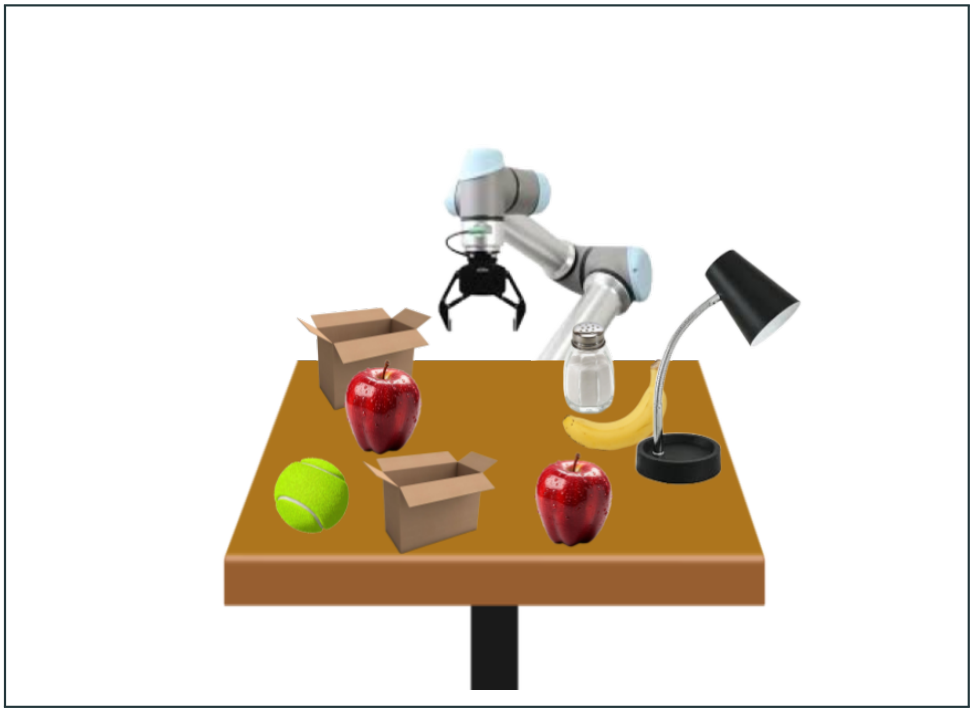
[2] alternatively: $\mathcal{O}(o|s) = p(O_t = o|S_t = s)$

Autonomy

Approaches to AI Decision Making

Formal Models

Best model for our setting?



From Single to Multi-Agent Settings

What changes in the **model** when we consider the other agents in the environment?



We will discuss this later on.

Summary

- Approaches for AI decision-making
- Formal models for each approach

What next ?

- Explore a robot's structure
- Take a deeper dive into the approaches in robotics

